# CORE – 14: Algorithm Design Techniques (Unit – 4)

**Representations of Graphs**: There are different ways to optimally represent a graph, depending on the density of its edges, type of operations to be performed and ease of use.

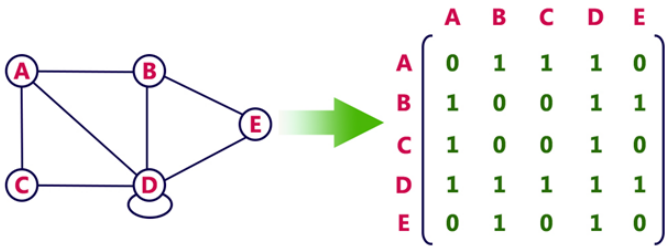
### 1. Adjacency Matrix

- Adjacency matrix is a sequential representation.
- It is used to represent which nodes are adjacent to each other. i.e. is there any edge connecting nodes to a graph.
- In this representation, we have to construct a n x n matrix A if the graph has n vertices. If there is any edge from a vertex i to vertex j, then the corresponding element of A, $a_{i,j} = 1$, otherwise $a_{i,j} = 0$.

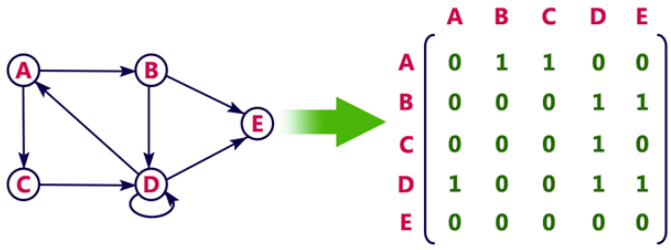
If there is any weighted graph then instead of 1s and 0s, we can store the weight of the edge.

Example:

Undirected graph representation

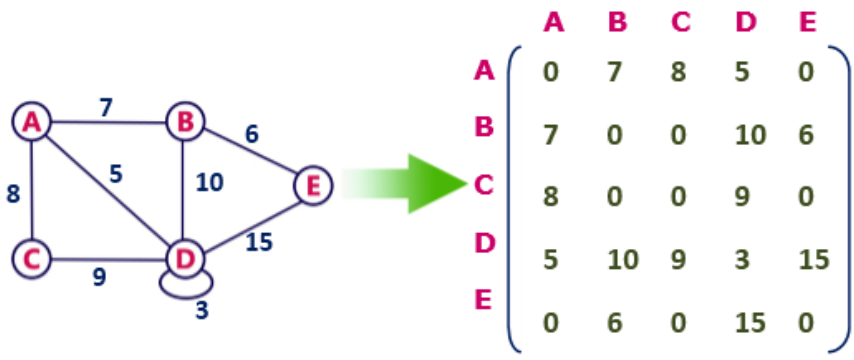



Directed graph represenation




In the above examples, 1 represents an edge from row vertex to column vertex, and 0 represents no edge from row vertex to column vertex.

**Undirected weighted graph represenation**

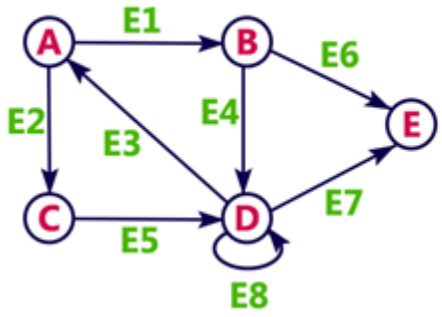



**2. Incidence Matrix:** In Incidence matrix representation, graph can be represented using a matrix of size: n x e where n = total number of vertices and e = total number of edges.

This matrix is filled with either **0 or 1** or -1. Where,

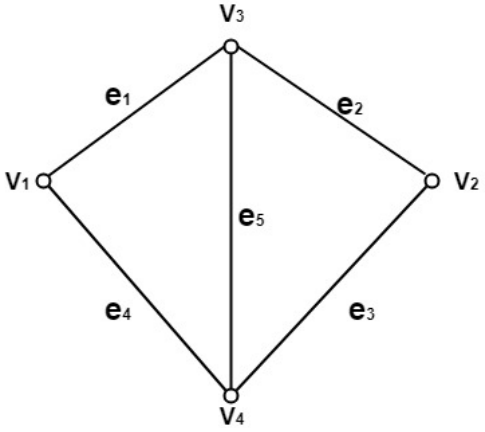
- 0 is used to represent row edge which is not connected to the vertex.
- 1 is used to represent row edge which is connected as outgoing edge to the vertex.
- -1 is used to represent row edge which is connected as incoming edge to the vertex.

Example of directed Graph:




| | E1 | E2 | E3 | E4 | E5 | E6 | E7 | E8 |
|---|---|---|---|---|---|---|---|---|
| A | 1 | 1 | -1 | 0 | 0 | 0 | 0 | 0 |
| B | -1 | 0 | 0 | 1 | 0 | 1 | 0 | 0 |
| C | 0 | -1 | 0 | 0 | 1 | 0 | 0 | 0 |
| D | 0 | 0 | 1 | -1 | -1 | 0 | 1 | 1 |
| E | 0 | 0 | 0 | 0 | 0 | -1 | -1 | 0 |

Example of undirected Graph:



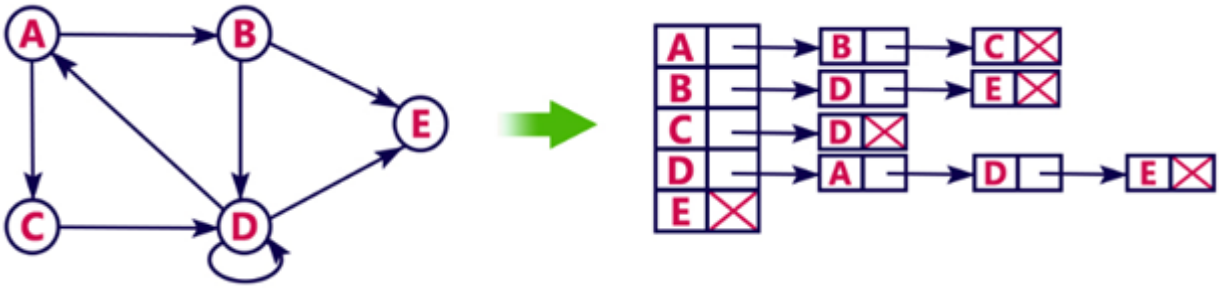

$M_I =$

| | $e_1$ | $e_2$ | $e_3$ | $e_4$ | $e_5$ |
|---|---|---|---|---|---|
| $V_1$ | 1 | 0 | 0 | 1 | 0 |
| $V_2$ | 0 | 1 | 1 | 0 | 0 |
| $V_3$ | 1 | 1 | 0 | 0 | 1 |
| $V_4$ | 0 | 0 | 1 | 1 | 1 |

## 3. Adjacency List

- Adjacency list is a linked representation.
- In this representation, for each vertex in the graph, we maintain the list of its neighbours. It means, every vertex of the graph contains list of its adjacent vertices.

Example

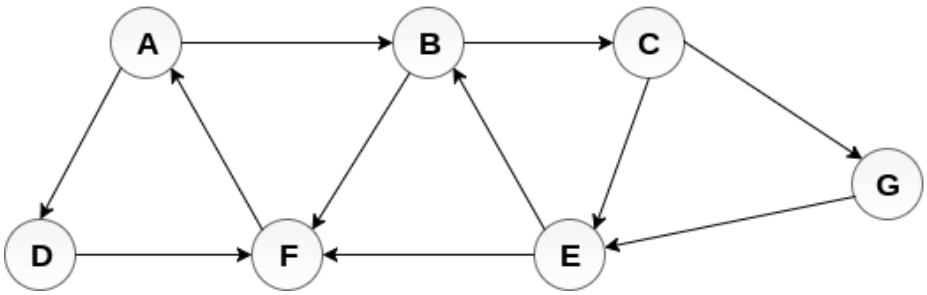


**Breadth-first search:** Breadth first search is a graph traversal algorithm that starts traversing the graph from root node and explores all the neighbouring nodes. Then, it selects the nearest node and explores all the unexplored nodes and ensures that each node is visited exactly once. The algorithm follows the same process for each of the nearest node until it finds the goal.

**Algorithm**

- **Step 1:** SET STATUS = 1 (ready state) for each node in G
- **Step 2:** Enqueue the starting node A and set its STATUS = 2 (waiting state)
- **Step 3:** Repeat Steps 4 and 5 until QUEUE is empty
- **Step 4:** Dequeue a node N. Process it and set its STATUS = 3 (processed state).
- **Step 5:** Enqueue all the neighbours of N that are in the ready state (whose STATUS = 1) and set their STATUS = 2(waiting state)
  [END OF LOOP]
- **Step 6:** EXIT

**Example**: Consider the graph G shown in the following figure; Find the path p from node A to node E using Breadth-first search.




**Adjacency Lists**

A : B, D
B : C, F
C : E, G
G : E
E : B, F
F : A
D : F

The algorithm uses two queues, namely **QUEUE1** and **QUEUE2**. **QUEUE1** holds all the nodes that are to be processed while **QUEUE2** holds all the nodes that are processed and deleted from **QUEUE1**.

**Lets start examining the graph from Node A.**

1. Add A to QUEUE1 and NULL to QUEUE2.
   QUEUE1 = {A}
   QUEUE2 = {NULL}

2. Delete the Node A from QUEUE1 and insert all its neighbours. Insert Node A into QUEUE2
   QUEUE1 = {B, D}
   QUEUE2 = {A}

3. Delete the node B from QUEUE1 and insert all its neighbours. Insert node B into QUEUE2.
   QUEUE1 = {D, C, F}
   QUEUE2 = {A, B}

4. Delete the node D from QUEUE1 and insert all its neighbours. Since F is the only neighbour of it which has been inserted, we will not insert it again. Insert node D into QUEUE2.

QUEUE1 = {C, F}
QUEUE2 = { A, B, D}

5. Delete the node C from QUEUE1 and insert all its neighbours. Add node C to QUEUE2.

QUEUE1 = {F, E, G}
QUEUE2 = {A, B, D, C}

6. Remove F from QUEUE1 and add all its neighbours. Since all of its neighbours have already been added, we will not add them again. Add node F to QUEUE2.

QUEUE1 = {E, G}
QUEUE2 = {A, B, D, C, F}

7. Remove E from QUEUE1, all of E's neighbours has already been added to QUEUE1 therefore we will not add them again. All the nodes are visited and the target node i.e. E is encountered into QUEUE2.

QUEUE1 = {G}
QUEUE2 = {A, B, D, C, F, E}

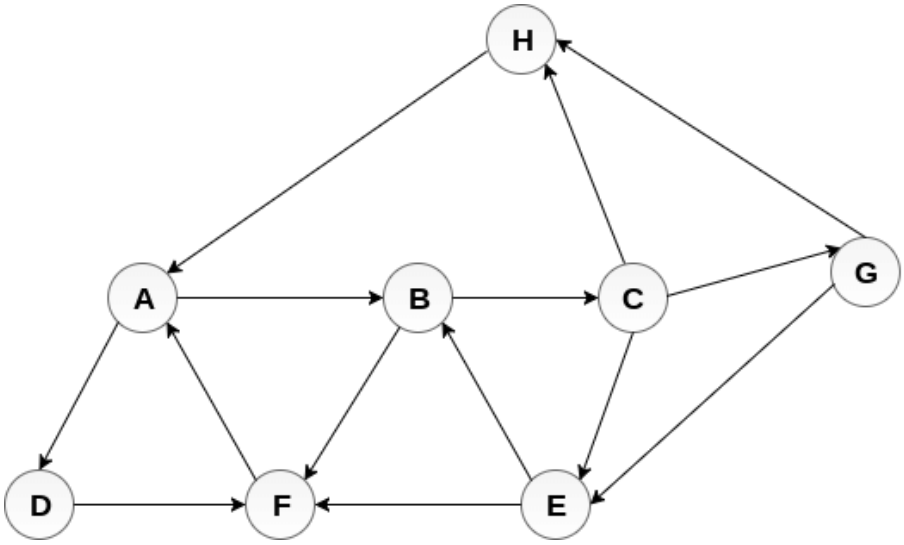
Now, backtrack from E to A, using the nodes available in QUEUE2. The path will be **A** → **B** → **C** → **E**.

**Depth-first search:** Depth first search (DFS) algorithm starts with the initial node of the graph G, and then goes to deeper and deeper until we find the goal node or the node which has no children. The algorithm, then backtracks from the dead end towards the most recent node that is yet to be completely unexplored. The data structure which is being used in DFS is **stack**.

**Algorithm**

- **Step 1:** SET STATUS = 1 (ready state) for each node in G
- **Step 2:** Push the starting node A on the stack and set its STATUS = 2 (waiting state)
- **Step 3:** Repeat Steps 4 and 5 until STACK is empty
- **Step 4:** Pop the top node N. Process it and set its STATUS = 3 (processed state)
- **Step 5:** Push on the stack all the neighbours of N that are in the ready state (whose STATUS = 1) and set their STATUS = 2 (waiting state)
  [END OF LOOP]
- **Step 6:** EXIT

**Example :** Consider the graph G along with its adjacency list, given in the figure below. Calculate the order to print all the nodes of the graph starting from node H, by using depth first search (DFS) algorithm.




**Adjacency Lists**

**A : B, D**
**B : C, F**
**C : E, G, H**
**G : E, H**
**E : B, F**
**F : A**
**D : F**
**H : A**

**Solution :**

Push H onto the stack
STACK : H

POP the top element of the stack i.e. H, print it and push all the neighbours of H onto the stack that are is ready state.
Print H STACK : A

Pop the top element of the stack i.e. A, print it and push all the neighbours of A onto the stack that are in ready state.
Print A Stack : B, D

Pop the top element of the stack i.e. D, print it and push all the neighbours of D onto the stack that are in ready state.
Print D Stack : B, F

Pop the top element of the stack i.e. F, print it and push all the neighbours of F onto the stack that are in ready state.
Print F Stack : B

Pop the top of the stack i.e. B and push all the neighbours
Print B Stack : C

Pop the top of the stack i.e. C and push all the neighbours.
Print C Stack : E, G

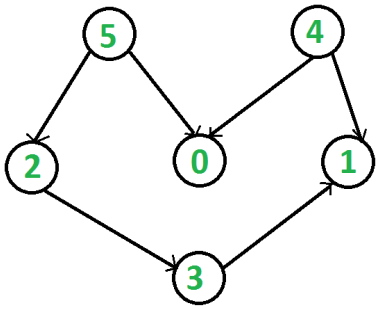
Pop the top of the stack i.e. G and push all its neighbours.
Print G Stack : E

Pop the top of the stack i.e. E and push all its neighbours.
Print E Stack :

Hence, the stack now becomes empty and all the nodes of the graph have been traversed.

The printing sequence of the graph will be : **H → A → D → F → B → C → G → E**

**Topological sort:** Topological sorting for Directed Acyclic Graph (DAG) is a linear ordering of vertices such that for every directed edge u v, vertex u comes before v in the ordering. Topological Sorting for a graph is not possible if the graph is not a DAG. It is mainly used for job scheduling, serialization etc.




For example, a topological sorting of the graph shown here is "5 4 2 0 3 1". There can be more than one topological sorting for a graph. For example, another topological sorting of the following graph is "4 5 2 0 3 1". The first vertex in topological sorting is always a vertex with in-degree as 0 (a vertex with no incoming edges).

**Algorithm:**

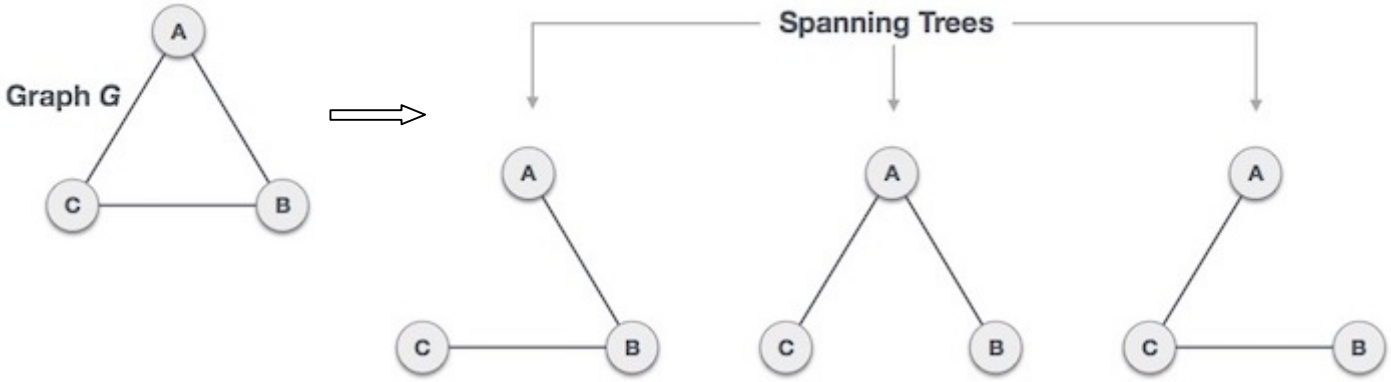
*L* ← Empty list that will contain the sorted elements
*S* ← Set of all nodes with no incoming edge
**while** *S* **is not** empty **do**
  remove a node *n* from *S and* add *the node* to *L*
  **for each** node *m* with an edge *e* from *n* to *m* **do**
    remove edge *e* from the graph
    **if** *m* has no other incoming edges **then**
      insert *m* into *S*
**if** *graph* has edges **then**
  **return** error *(graph has at least one cycle)*
**else**
  **return** *L* *(a topologically sorted order)*

## Minimum Spanning Trees:

**Tree**: A tree is a graph with the following properties:

1. The graph is connected (can go from anywhere to anywhere)
2. There are no cyclic (Acyclic)

**Spanning Tree**: Given a connected undirected graph, a spanning tree of that graph is a subgraph that is a tree and joined all vertices. A single graph can have many spanning trees. For Example-



**Properties of Spanning Tree:**

- A connected graph G can have more than one spanning tree.
- All possible spanning trees of graph G, have the same number of edges and vertices.
- The spanning tree does not have any cycle (loops).
- Removing one edge from the spanning tree will make the graph disconnected, i.e. the spanning tree is **minimally connected**.
- Adding one edge to the spanning tree will create a circuit or loop, i.e. the spanning tree is **maximally acyclic**.
- Spanning tree has **n-1** edges, where **n** is the number of nodes (vertices).

**Minimum Spanning Tree**: In a weighted graph, a minimum spanning tree is a spanning tree that has minimum weight than all other spanning trees of the same graph. In real-world situations, this weight can be measured as distance, congestion, traffic load or any arbitrary value denoted to the edges.

**Application of Minimum Spanning Tree:**

1. Designing Local Area Networks.
2. Laying pipelines connecting offshore drilling sites, refineries and consumer markets.
3. Connecting Telephone Wire to different houses/offices in a citiy.

**Minimum Spanning-Tree Algorithm**: The two most important spanning tree algorithms are −

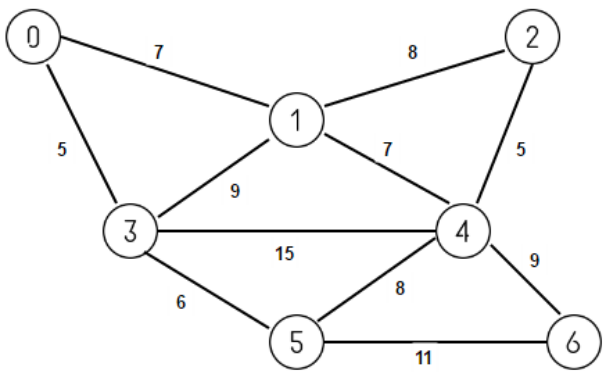
- Kruskal's Algorithm
- Prim's Algorithm

Both are greedy algorithms.

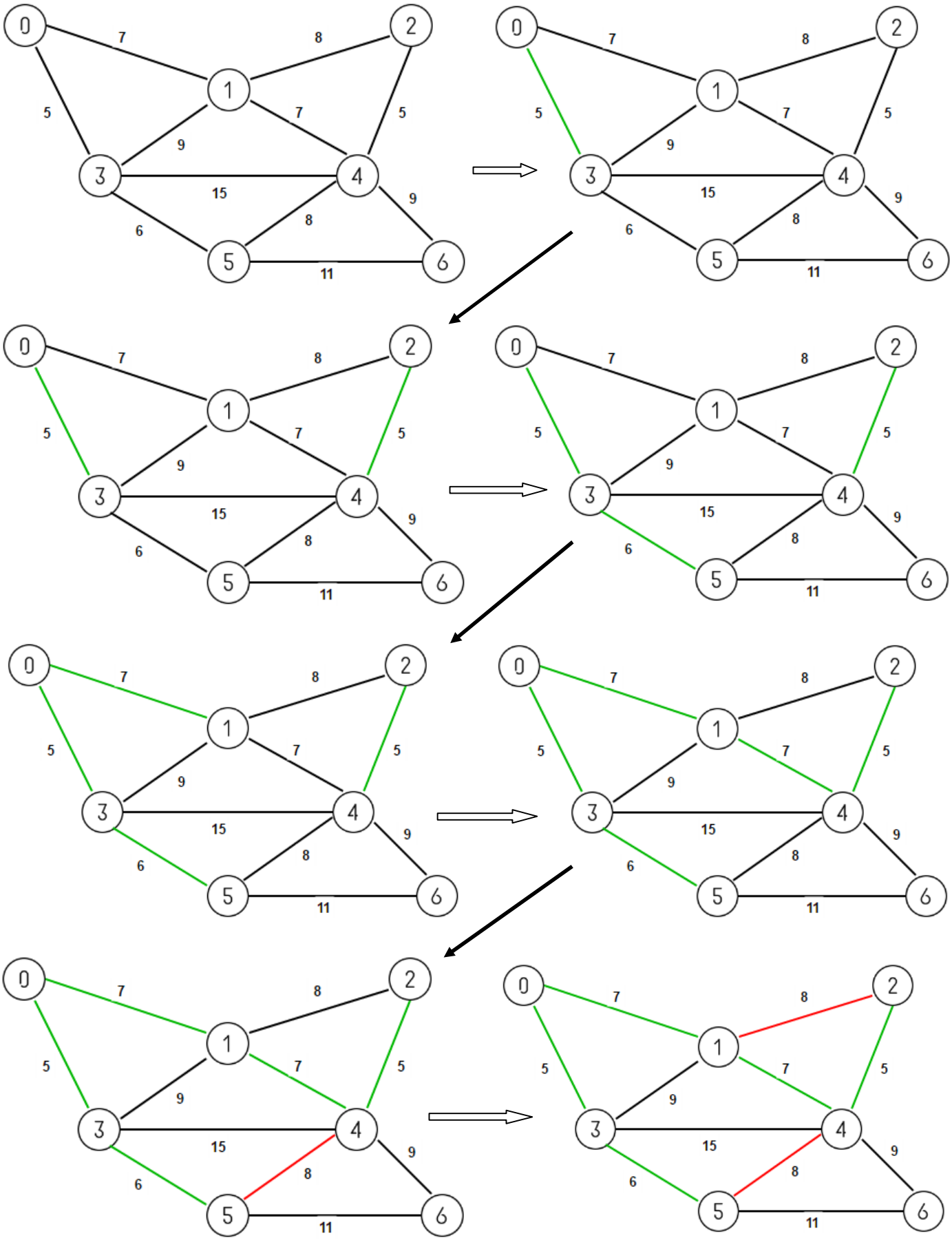
## Kruskal's Algorithm:

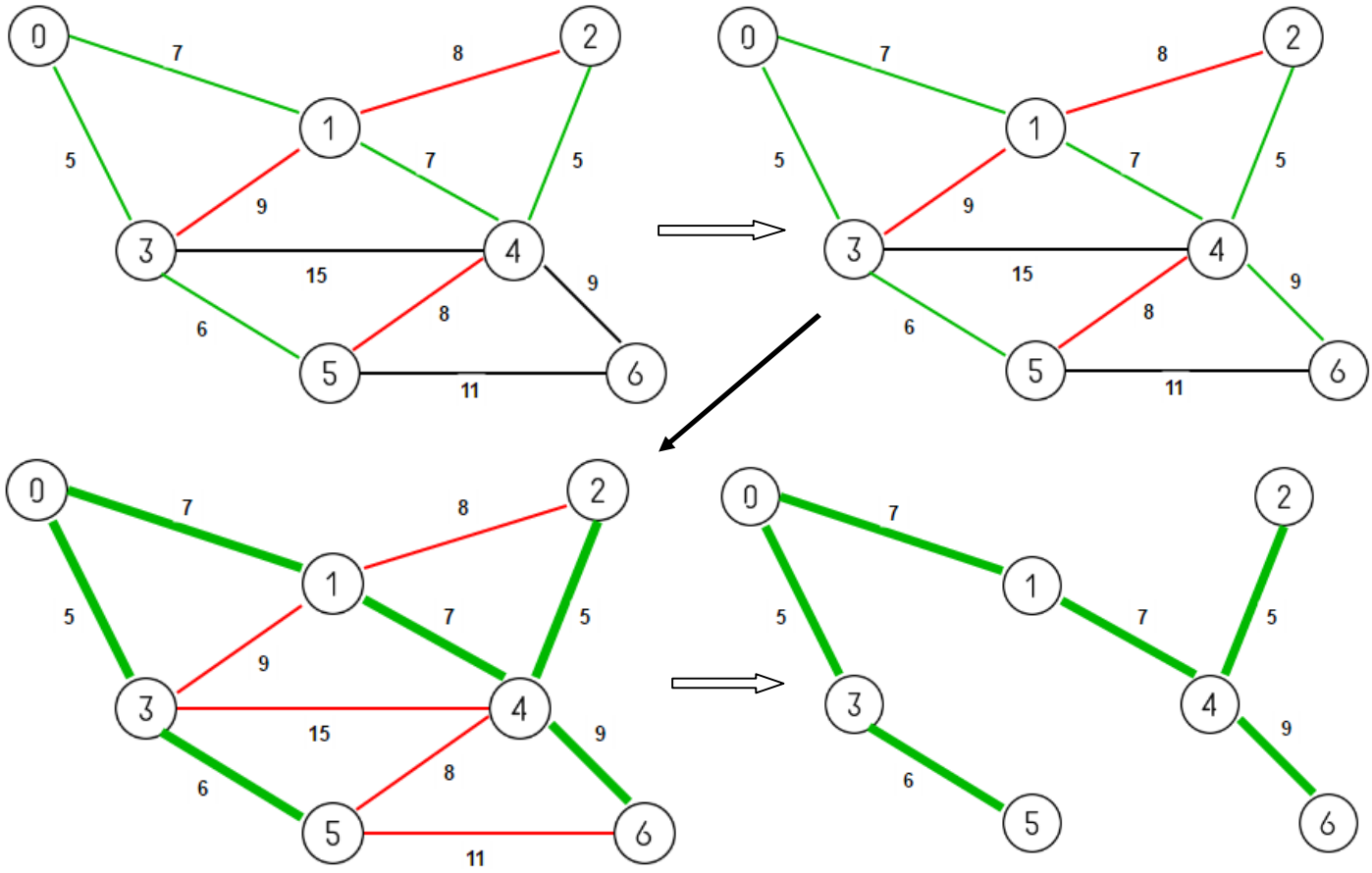
Sort all edges in graph G in order of their increasing weights;
repeat V-1 times // as MST contains V-1 edges
{
  select the next edge with minimum weight from graph G;
  if (no cycle is formed by adding the edge in MST, i.e., the edge connects two different connected components in MST)
    add the edge to MST;
}

**Example:** Draw the minimum spanning tree of the given undirected connected weighted graph using Kruskal's Algorithm.



**Solution:**

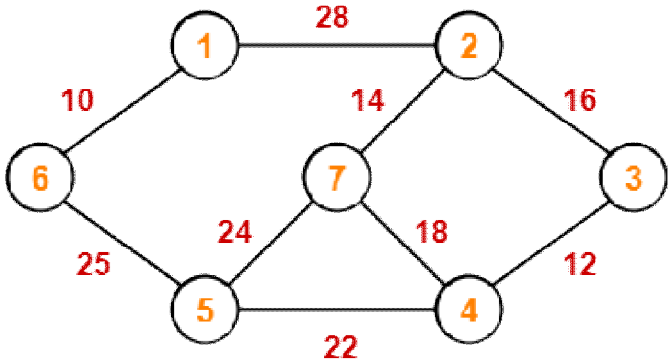




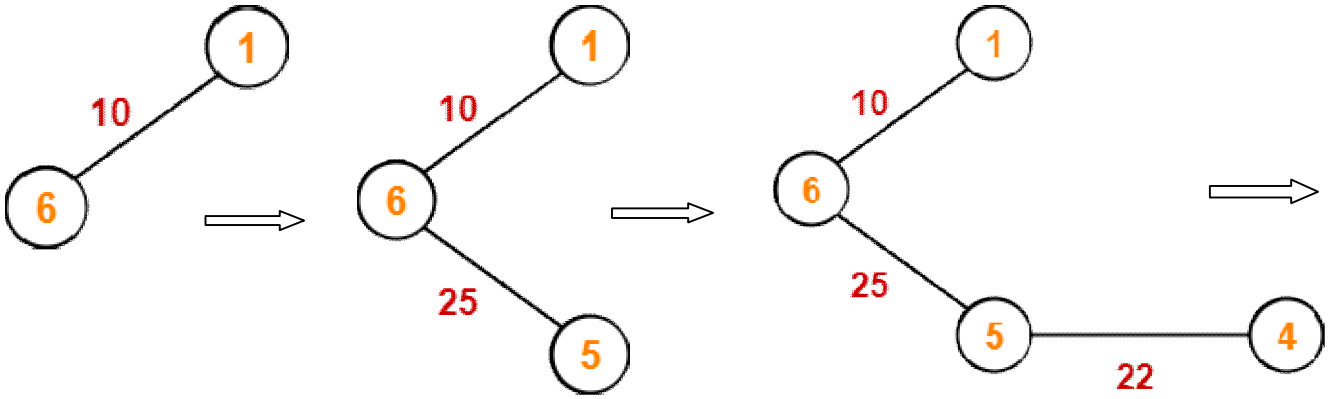

## Prim's algorithm:

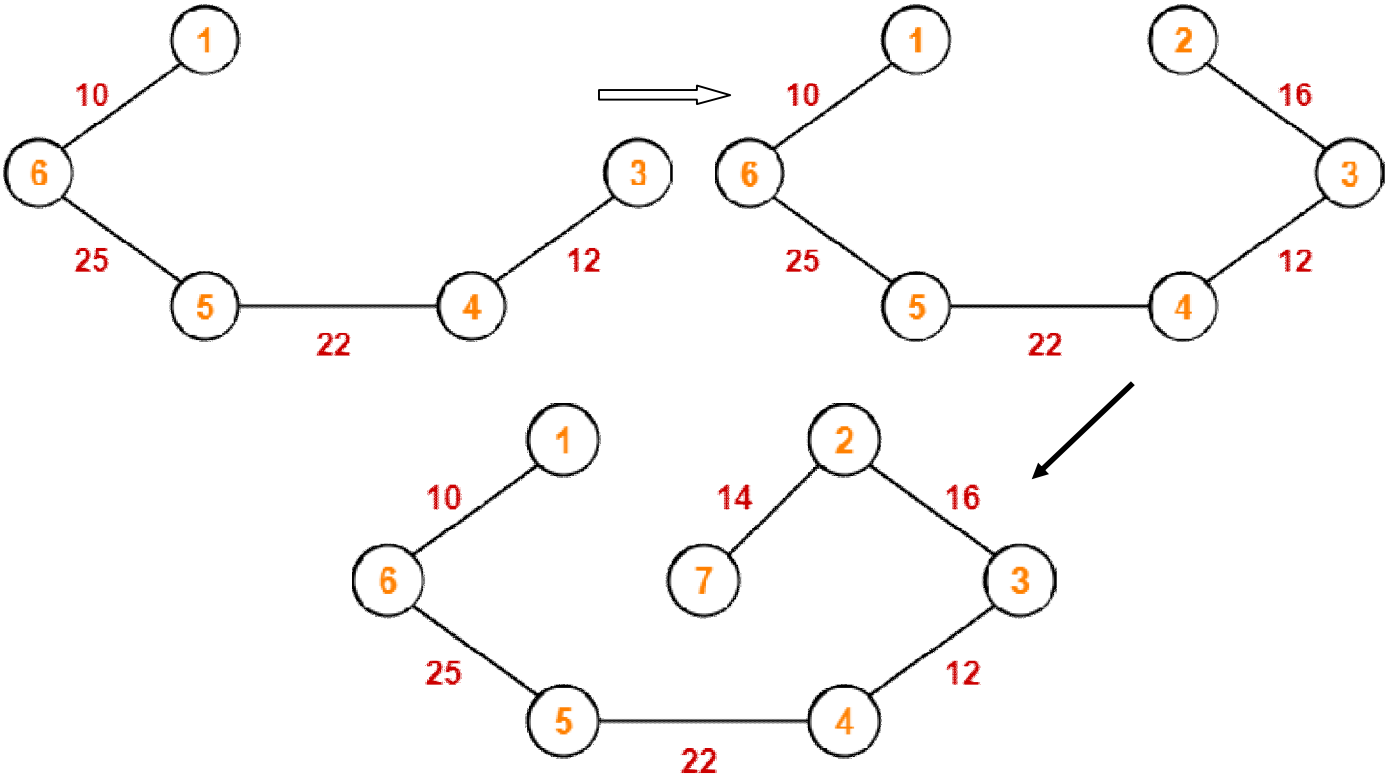
1. Initialize a tree with a single vertex, chosen arbitrarily from the graph.
2. Grow the tree by one edge: of the edges that connect the tree to vertices not yet in the tree, find the minimum-weight edge, and transfer it to the tree.
3. Repeat step 2 (until all vertices are in the tree).

**Example**: Construct the minimum spanning tree (MST) for the given graph using Prim's Algorithm




**Solution:**






**Single-source shortest paths:** The single-source shortest path algorithm means that, given a weighted graph, the algorithm will find the **shortest distance from a selected node to all other nodes**. Given a graph G = (V, E), we maintain for each vertex v ∈ V a **predecessor π[v]** that is either another vertex or NIL. The single source shortest paths are based on a technique known as **relaxation**, a method that repeatedly decreases an upper bound on the actual **shortest path weight of each vertex** until the upper bound equivalent the shortest path weight. For each vertex v ∈ V, we maintain an attribute d[v], which is an upper bound on the weight of the shortest path from source s to v. We call d[v] the **shortest path estimate**.

**INITIALIZE - SINGLE - SOURCE (G, s)**

1. for each vertex v ∈ V[G]
2. do d[v] ← ∞
3. π[v] ← NIL
4. d[s] ← 0

After initialization, π[v] = NIL ∀ v ∈ V, d[v] = 0 for v = s, and d[v] = ∞ for v ∈ V - {s}.

The development of relaxing an edge (u, v) consists of testing whether we can improve the shortest path to v found so far by going through u and if so, updating d[v] and π[v]. A relaxation step may decrease the value of the shortest - path estimate d[v] and updated v's predecessor field π[v].

**RELAX (u, v, w)**

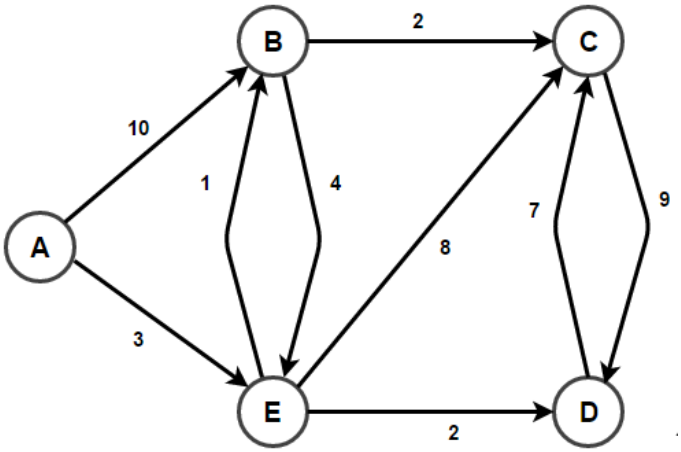
1. If d[v] > (d[u] + w(u, v))
   then d[v] ← d[u] + w(u, v)
   π[v] ← u

**Dijkstra's Algorithm**: It is a greedy algorithm that solves the single-source shortest path problem for a directed graph G = (V, E) with nonnegative edge weights, i.e., w (u, v) ≥ 0 for each edge (u, v) ∈ E.

Dijkstra's Algorithm maintains a set S of vertices whose final shortest - path weights from the source s have already been determined. That's for all vertices v ∈ S; we have d[v] = δ(s, v). The algorithm repeatedly selects the vertex u ∈ V - S with the minimum shortest path estimate, inserts u into S and relaxes all edges leaving u.

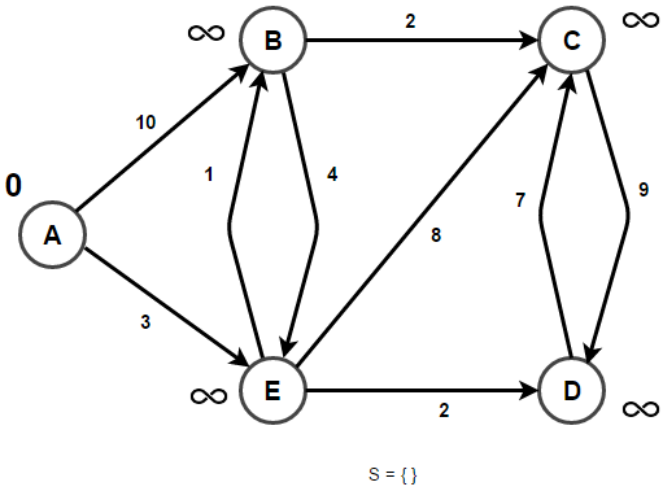
Because it always chooses the "lightest" or "closest" vertex in V - S to insert into set S, it is called as the **greedy strategy**.

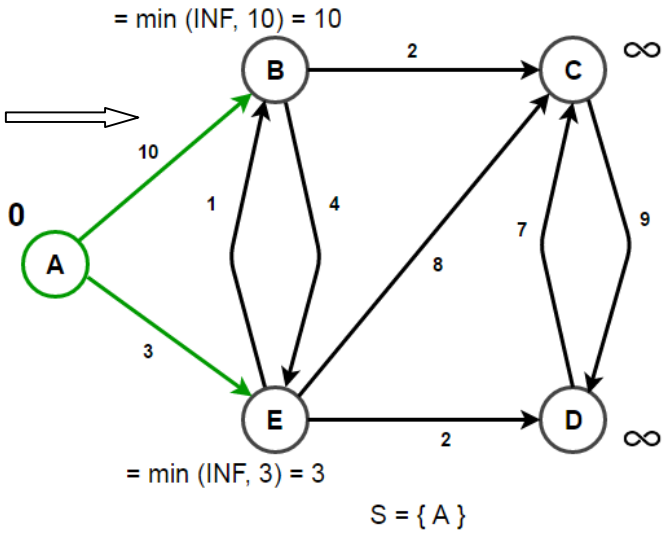
**Example:** Here is a directed weighted graph. Find the shortest path from node A to all other nodes using Dijkstra's algorithm.




**Solution:** Initially S contains the source vertex. So, S = {A}. The vertex A has a distance 0 and all other vertex has distance ∞ from the source.

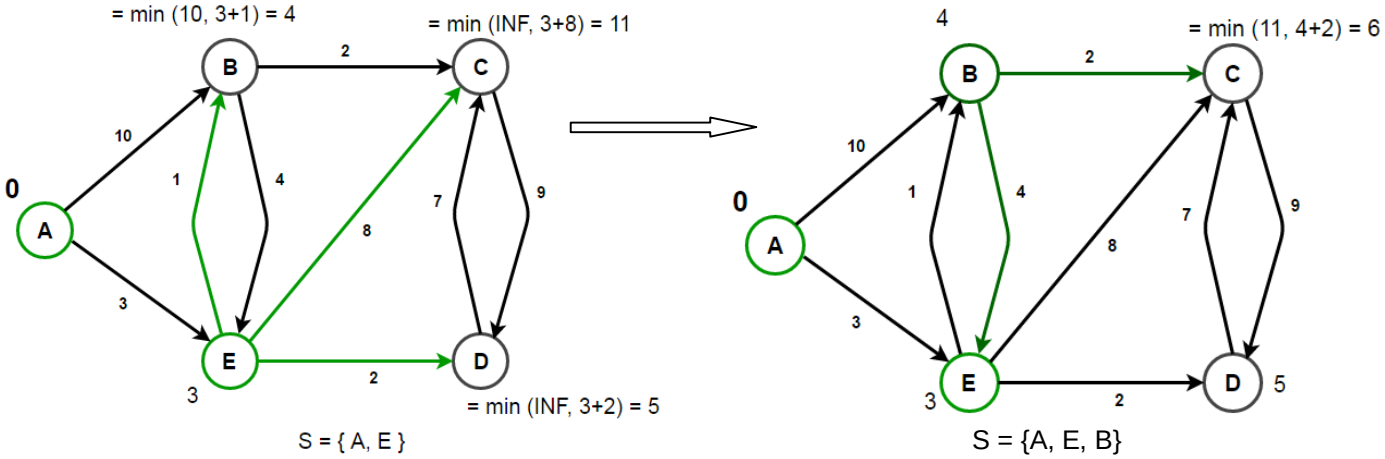
We start from the source vertex A and start relaxing A's neighbours. Since Vertex B can be reached through a direct edge from vertex A, we update its distance from ∞ to 10 (weight of edge AB). Similarly we can reach vertex E from vertex A through a direct edge, so we update the distance from ∞ to 3.



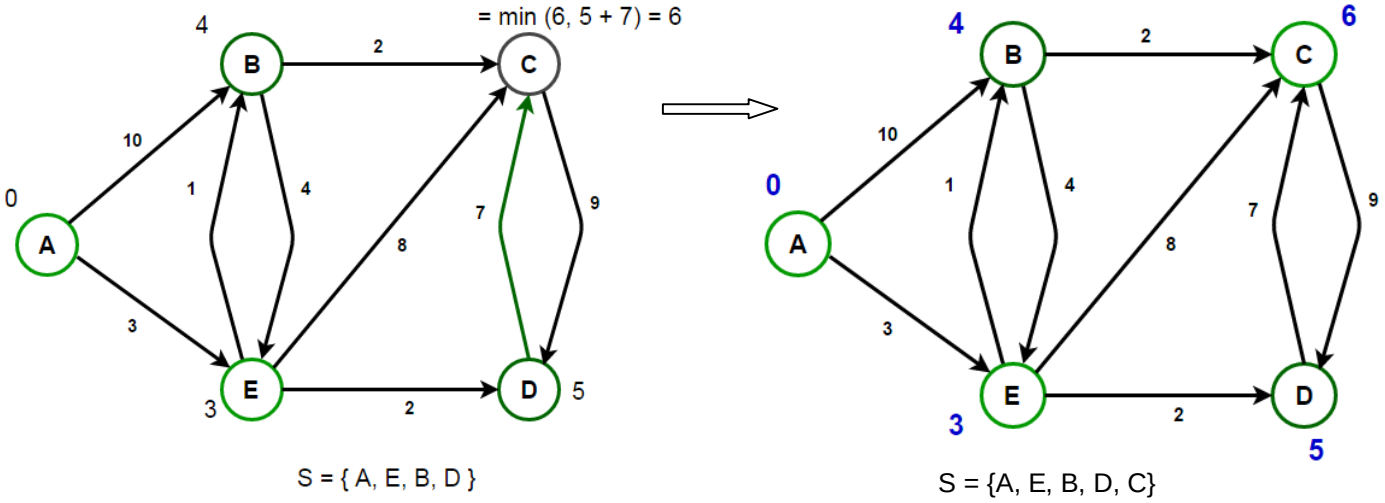




After processing all outgoing edges of A, we next consider a vertex with minimum distance. So we chose E and insert it into set S. Now set S becomes S = {A, E}.

Next we relax all the neighbours of E i.e. B, C and D and chose the vertex which has minimum cost from the list Q. So the vertex B is removed from Q and is inserted into the set S. So set S becomes S = {A, E, B}.




Next we relax all the neighbours of B which are not in set S. We chose the vertx which has minimum cost from the set Q. So the vetex D is chosen and is inserted into set S. We repeat this process untill we relax all the nodes and Q becomes empty.




The above process can be summarized in a tabular form as follows.

| Iteration | d[A] | d[B] | d[C] | d[D] | d[E] | Set S = | Set Q = | Vertex Selected |
|---|---|---|---|---|---|---|---|---|
| 0 | 0 | ∞ | ∞ | ∞ | ∞ | { } | {A, B, C, D, E} | A |
| 1 | 0 | 10 | ∞ | ∞ | 3 | {A} | {B, C, D, E} | E |
| 2 | 0 | 4 | 11 | 5 | 3 | {A, E} | {B, C, D} | B |
| 3 | 0 | 4 | 6 | 5 | 3 | {A, E, B} | {C, D} | D |
| 4 | 0 | 4 | 6 | 5 | 3 | {A, E, B, D} | {C} | C |
| 5 | **0** | **4** | **6** | **5** | **3** | {A, E, B, D, C} | { } | |

Finally, we find the minimum cost from source A to all other vertices as shown in the above table..

**Dijkstra's Algorithm (G, w, s)**

1. INITIALIZE - SINGLE - SOURCE (G, s)
2. S ← ∅
3. Q ← V [G]
4. While Q ≠ ∅
5. Do u ← EXTRACT - MIN (Q)
6. S ← S ∪ {u}
7. For each vertex v ∈ Adj [u]
8. Do RELAX (u, v, w)

**// EXTRACT - MIN (Q) find the node of minimum distance from source and deletes the node from the list 'Q'.**

The time complexity of Dijkstra's algorithm is **$O(V^2)$.** The main disadvantage of Dijkstra's algorithm is it cannot handle negative edges and most often cannot obtain the right shortest path.

**Bellman-Ford algorithm**: Bellman-Ford Algorithm solves single source shortest path problem in which edge weight may be negative but no negative cycle exists. It is slower than Dijkstra's Algorithm but more versatile, as it capable of handling some of the negative weight edges. This algorithm detects the negative cycle in a graph and reports their existence.

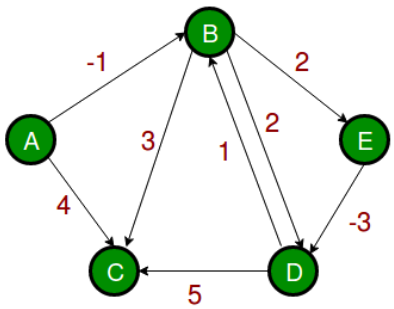
Based on the "Principle of Relaxation" in which more accurate values are gradually recovered.

Given a weighted directed graph G = (V, E) with source s and weight function w: E → R, the Bellman-Ford algorithm returns a Boolean value indicating whether or not there is a negative weight cycle. If there is no such a cycle, the algorithm produces the shortest paths and their weights. The algorithm returns **TRUE** if and only if a graph contains **no negative - weight cycles** that are reachable from the source.

**BELLMAN -FORD (G, w, s)**

1. INITIALIZE - SINGLE - SOURCE (G, s)
2. for i ← 1 to |V[G]| - 1
3. do for each edge (u, v) ∈ E [G]
4. do RELAX (u, v, w)
5. for each edge (u, v) ∈ E [G]
6. do if d [v] > d [u] + w (u, v)
7. then return FALSE.
8. return TRUE.

**Example:** For the graph shown here, Find shortest paths from the node A to all other nodes using Bellman Ford Algorithm.



**Solution**: Initialize all distances as infinite, except the distance to the source itself. Total number of vertices in the graph is 5, so all edges must be processed 4 times. Let the edges are processed in the following order:

(B, E), (D, B), (B, D), (A, B), (A, C), (D, C), (B, C), (E, D).

The following table shows the pupation of distances of all the nodes from the source node.

| Iteration | Edge relaxed | Distance from source vertex A to all other vertx | | | | |
|---|---|---|---|---|---|---|
| | | A | B | C | D | E |
| Initialization | - | 0 | ∞ | ∞ | ∞ | ∞ |
| Iteration-1 | (B, E) | 0 | ∞ | ∞ | ∞ | ∞ |
| | (D, B) | 0 | ∞ | ∞ | ∞ | ∞ |
| | (B, D) | 0 | ∞ | ∞ | ∞ | ∞ |
| | (A, B) | 0 | -1 | ∞ | ∞ | ∞ |
| | (A, C) | 0 | -1 | 4 | ∞ | ∞ |
| | (D, C) | 0 | -1 | 4 | ∞ | ∞ |
| | (B, C) | 0 | -1 | 2 | ∞ | ∞ |
| | (E, D) | 0 | -1 | 2 | ∞ | ∞ |
| Iteration-2 | (B, E) | 0 | -1 | 2 | ∞ | 1 |
| | (D, B) | 0 | -1 | 2 | ∞ | 1 |
| | (B, D) | 0 | -1 | 2 | 1 | 1 |
| | (A, B) | 0 | -1 | 2 | 1 | 1 |
| | (A, C) | 0 | -1 | 2 | 1 | 1 |
| | (D, C) | 0 | -1 | 2 | 1 | 1 |
| | (B, C) | 0 | -1 | 2 | 1 | 1 |
| | (E, D) | 0 | -1 | 2 | -2 | 1 |

The algorithm processes all edges 2 more times. The distances are minimized after the second iteration, so third and fourth iterations don't update the distances. Time complexity of Bellman Ford's Algorithm is O (VE).

**Bellman Ford's Algorithm vs. Dijkstra's Algorithm**

| Bellman Ford's Algorithm | Dijkstra's Algorithm |
|---|---|
| Bellman Ford's Algorithm works when there is negative weight edge, it also detects the negative weight cycle. | Dijkstra's Algorithm doesn't work when there is negative weight edge. |
| It is more time consuming than Dijkstra's algorithm. Its time complexity is O (VE). In the worst case it is O ($V^3$). | It is less time consuming. The time complexity is O ($V^2$). |
| It is dynamic programming approach based algorithm. | It is Greedy approach based algorithm. |